

# **Week 4 - Secure Encryption Tool for Data Storage**

## **ABSTRACT**

Data security is a major concern in modern computing environments. Sensitive information must be protected from unauthorized access during storage and transmission. Encryption is one of the most effective methods used to ensure data confidentiality.

This project implements a Secure Encryption Tool using Python and the Cryptography library. The tool encrypts files using the Fernet encryption method, which is based on AES symmetric encryption. The encrypted file can only be accessed by decrypting it with the correct encryption key.

The system demonstrates how cryptographic techniques can be applied to protect sensitive data stored on a system.

## **INTRODUCTION**

With the increasing amount of sensitive data stored digitally, protecting information from unauthorized access has become extremely important. Encryption provides a secure method of protecting data by converting readable information into an unreadable format.

Only authorized users possessing the correct key can decrypt and access the original data. Encryption is widely used in cybersecurity applications including secure file storage, secure communication, and password protection.

The Secure Encryption Tool developed in this project demonstrates a simple implementation of file encryption and decryption using Python.

## **TOOLS AND TECHNOLOGIES USED**

1. **Python Programming Language:** Python was used to implement the encryption and decryption logic.

2. **Cryptography Library:** The cryptography library provides secure encryption methods. The project uses the Fernet module which implements AES-based symmetric encryption.
3. **File Handling:** Python file handling was used to read and write encrypted and decrypted files.

## **STEPS INVOLVED IN BUILDING THE PROJECT**

### **Step 1: Key Generation**

A secure encryption key is generated using the Fernet module and stored securely for future decryption.

### **Step 2: File Encryption**

The input file is read in binary format and encrypted using the generated key.

### **Step 3: Encrypted File Storage**

The encrypted data is stored as a binary file which cannot be read without decryption.

### **Step 4: File Decryption**

When the user chooses to decrypt, the encryption key is loaded and used to restore the original file content.

### **Step 5: Secure Output Generation**

The decrypted file is saved in text format, restoring the original data.

## **CONCLUSION:**

The Secure Encryption Tool demonstrates how cryptographic techniques can be applied to protect sensitive data stored on a system. By encrypting files before storage, unauthorized users cannot access the information without the correct key.

Although this project is a simplified implementation, it reflects the core concepts used in real-world cybersecurity systems. Future improvements could include password-protected keys, secure key management systems, and graphical user interfaces for easier interaction.